

Prova in itinere Elettronica

Aprile 2020

Domande di teoria: rispondere ad entrambe le domande

Progetto: sviluppare la macchina a stati e completare il diagramma temporale

$ab \setminus cd$	00	01	11	10
00	0	0	1	1
01	1	0	0	1
11	1	0	1	1
10	1	1	0	0

$ab \setminus cd$	00	01	11	10
00	-	0	1	-
01	1	0	0	-
11	1	-	1	-
10	-	1	0	0

1. Semplificazione logica (6 pts)

- Si considerino le mappe di Karnaugh mostrate in figura. Quella di sinistra è completamente specificata, mentre quella di destra è derivata dalla prima con l'aggiunta di don't care
- **Per entrambe le mappe:**
 - individuare tutti gli implicant primari, mostrando la loro espressione
 - indicare quali implicant primari sono essenziali e perché
 - derivare una copertura minima
- Confrontare quindi la complessità dei due risultati minimi in termini di numero di letterali

$ab \backslash cd$	00	01	11	10
00	0	0	1	1
01	1	0	0	1
11	1	0	1	1
10	1	1	0	0

$ab \backslash cd$	00	01	11	10
00	-	0	1	-
01	1	0	0	-
11	1	-	1	-
10	-	1	0	0

1. Soluzione

Prima mappa. Gli implicanti primi sono: bd' abc $a'b'c$ $ab'c'$ $a'cd'$ $ac'd'$

Un implicante primo è essenziale se copre un uno non coperto da nessun altro implicante primo. Gli implicanti primi essenziali sono quindi: bd' , perché copre 0100, abc perché copre 1111, $a'b'c$ perché copre 0011, $ab'c'$ perché copre 1001.

L'espressione minima è costituita dai soli implicanti primi essenziali: $bd' + abc + a'b'c + ab'c'$

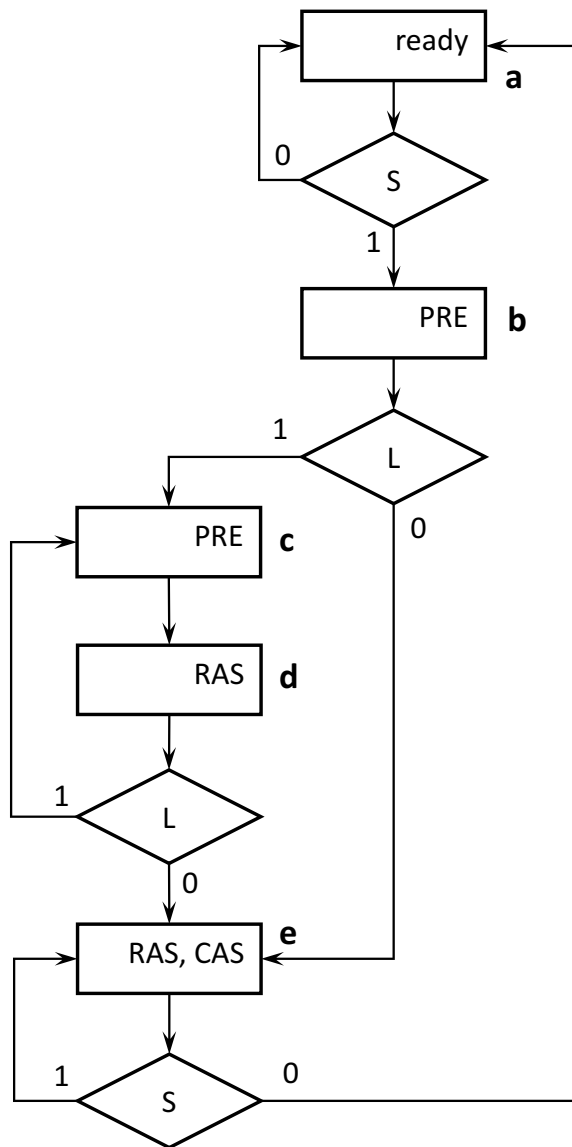
Seconda mappa. Gli implicanti primi sono: $a'd'$ bd' ab ac' $a'b'c$ $c'd'$

Gli implicanti primi essenziali sono: ab perché copre 1111, ac' perché copre 1001, $a'b'c$ perché copre 0011

Vi sono diverse possibili espressioni minime che includono gli implicanti primi essenziali più un implicante per coprire 0100, tutti della stessa dimensione:

$ab + ac' + a'b'c + a'd'$, $ab + ac' + a'b'c + c'd'$, $ab + ac' + a'b'c + bd'$

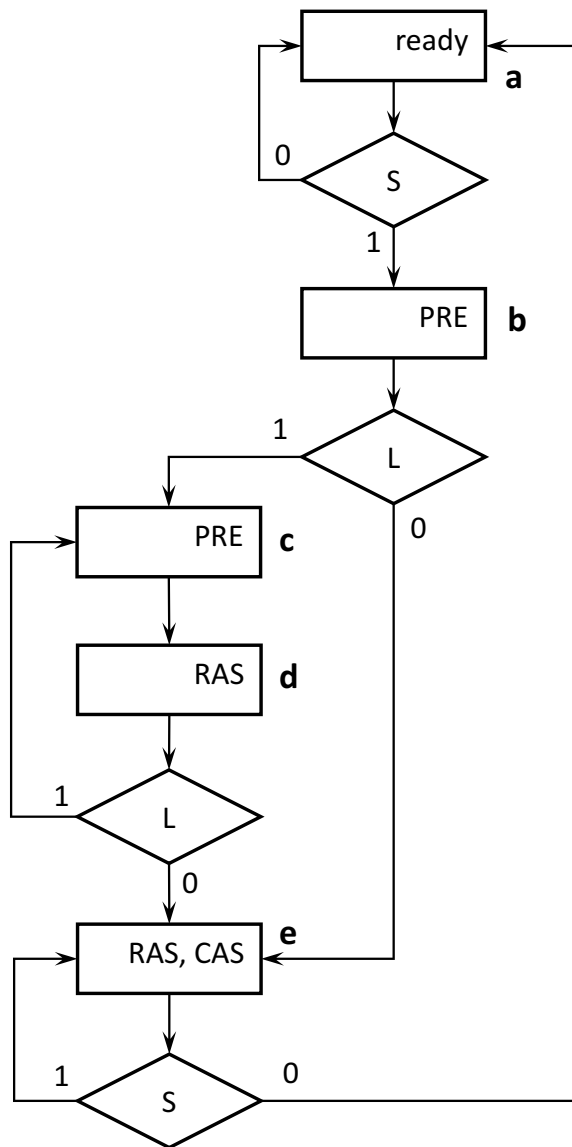
Confrontando le due espressioni si vede che la mappa con i don't care usa 9 invece di 11 letterali, e risulta quindi in un circuito più piccolo.



2. Sintesi di macchina a stati (6 pts)

- Si consideri la macchina a stati mostrata nella figura a fianco
- Vi sono cinque stati: **a, b, c, d, e**
- Vi sono due ingressi: **S** ed **L**
- Vi sono quattro uscite: **ready, PRE, RAS, CAS**
- Sintetizzare un circuito che implementi la macchina a stati **utilizzando la codifica one-hot**. In particolare:
 - Si spieghi brevemente in cosa consiste la codifica one-hot
 - Si derivino le espressioni per il calcolo dello stato futuro (a_{+1} , b_{+1} , etc.) e quelle per il calcolo delle uscite
 - Si scriva la tabella di calcolo dello stato futuro e delle uscite *limitatamente agli stati raggiungibili*. (suggerimento: usare un trattino per indicare un ingresso che non influisce sulla transizione)
- Si disegni il circuito

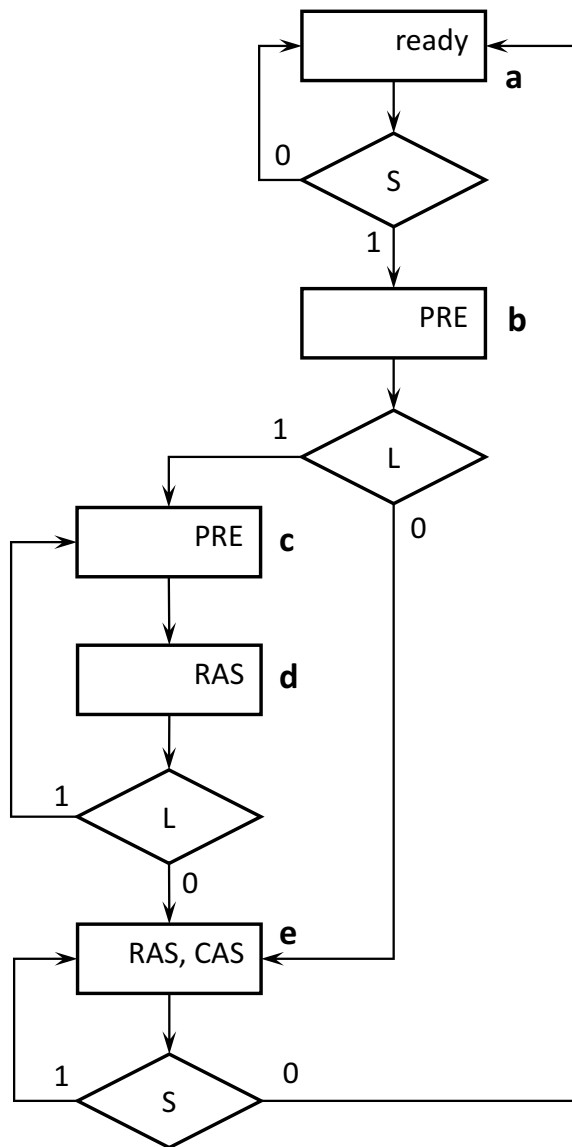
Non è necessario usare mappe di Karnaugh (ma non è vietato usarle)



2. Soluzione

La codifica one-hot consiste nell'utilizzare una variabile di stato per ogni stato, codificate in modo che una sola variabile sia a 1, mentre le altre sono tutte a 0. In questo modo lo stato è già decodificato. Nel nostro caso saranno necessarie 5 variabili di stato, che chiamiamo a, b, c, d, e. La codifica è la seguente:

Stato	a	b	c	d	e
a	1	0	0	0	0
b	0	1	0	0	0
c	0	0	1	0	0
d	0	0	0	1	0
e	0	0	0	0	1



2. Soluzione

Le espressioni per il calcolo dello stato futuro si ottengono facilmente guardando le transizioni in entrata in ciascuno stato, e considerando lo stato di partenza e le eventuali condizioni sugli ingressi. Per il nostro caso otteniamo:

$$a_{+1} = aS' + eS' = S'(a + e)$$

$$b_{+1} = aS$$

$$c_{+1} = bL + dL = L(b + d)$$

$$d_{+1} = c$$

$$e_{+1} = bL' + dL' + eS = L'(b + d) + eS$$

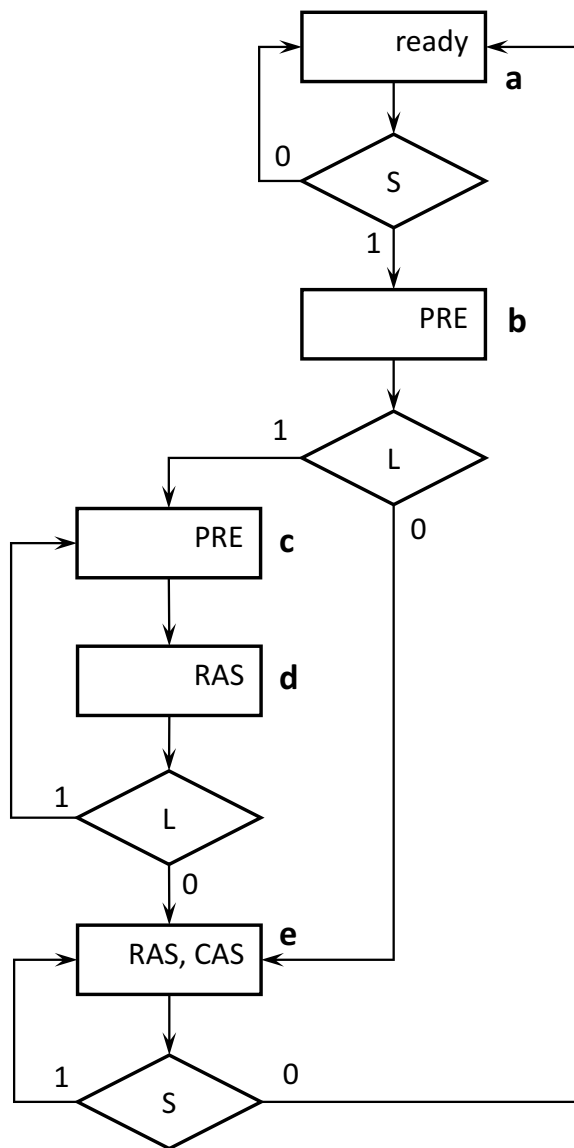
Per le uscite, trattandosi di una macchina di Moore, è sufficiente mettere in or le variabili degli stati in cui l'uscita è attiva. Si ottiene:

$$\text{ready} = a$$

$$\text{PRE} = b + c$$

$$\text{RAS} = d + e$$

$$\text{CAS} = e$$



2. Soluzione

La tabella del calcolo dello stato futuro e delle uscite si ottiene facilmente dalle espressioni derivate prima, o semplicemente dalla macchina a stati:

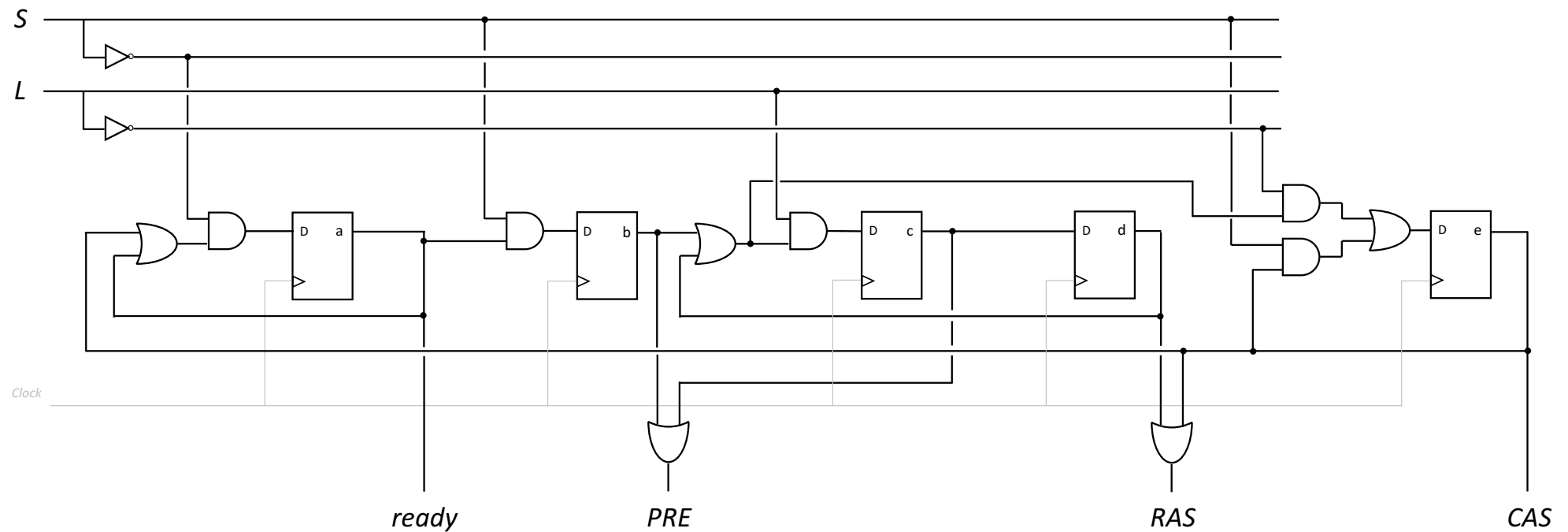
Stato pres.					Ing.		Stato futuro					Uscite			
a	b	c	d	e	S	L	a ₊₁	b ₊₁	c ₊₁	d ₊₁	e ₊₁	ready	PRE	RAS	CAS
1	0	0	0	0	0	-	1	0	0	0	0	1	0	0	0
1	0	0	0	0	1	-	0	1	0	0	0	1	0	0	0
0	1	0	0	0	-	0	0	0	0	0	1	0	1	0	0
0	1	0	0	0	-	1	0	0	1	0	0	0	1	0	0
0	0	1	0	0	-	-	0	0	0	1	0	0	1	0	0
0	0	0	1	0	-	0	0	0	0	0	1	0	0	1	0
0	0	0	1	0	-	1	0	0	1	0	0	0	0	1	0
0	0	0	0	1	0	-	1	0	0	0	0	0	0	1	1
0	0	0	0	1	1	-	0	0	0	0	1	0	0	1	1

2. Soluzione

Il circuito ricalca con le porte logiche le espressioni derivate precedentemente. Si sono usati flip flop edge triggered di tipo D per gli elementi di memoria.

2. Soluzione

Il circuito ricalca con le porte logiche le espressioni derivate precedentemente. Si sono usati flip flop edge triggered di tipo D per gli elementi di memoria.



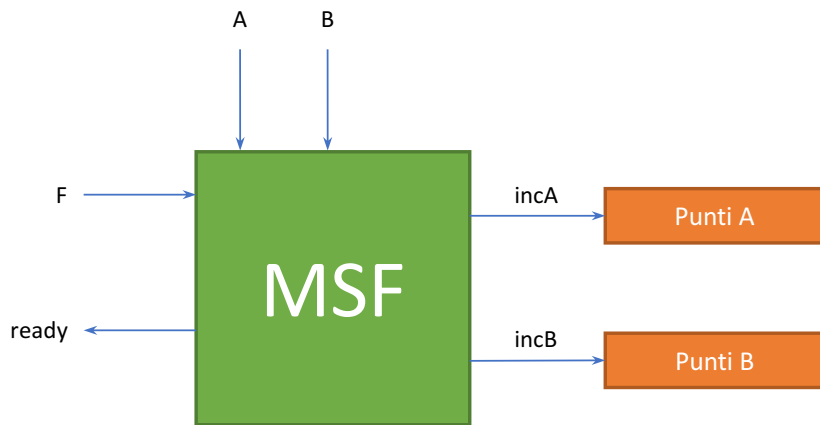
3. Progetto

Si richiede di

- (13 pts) Progettare la macchina a stati finiti mostrando e *commentando* il diagramma degli stati
- (5 pts) Completare il diagramma temporale allegato

The ReFlex Game

Provetta Aprile 2020



Sistema

- I contatori **Punti A** e **Punti B** sono contatori sincroni. Vengono automaticamente inizializzati a 0 all'inizio del gioco. I segnali **incA** e **incB** sono segnali sincroni di abilitazione al conteggio.

Note

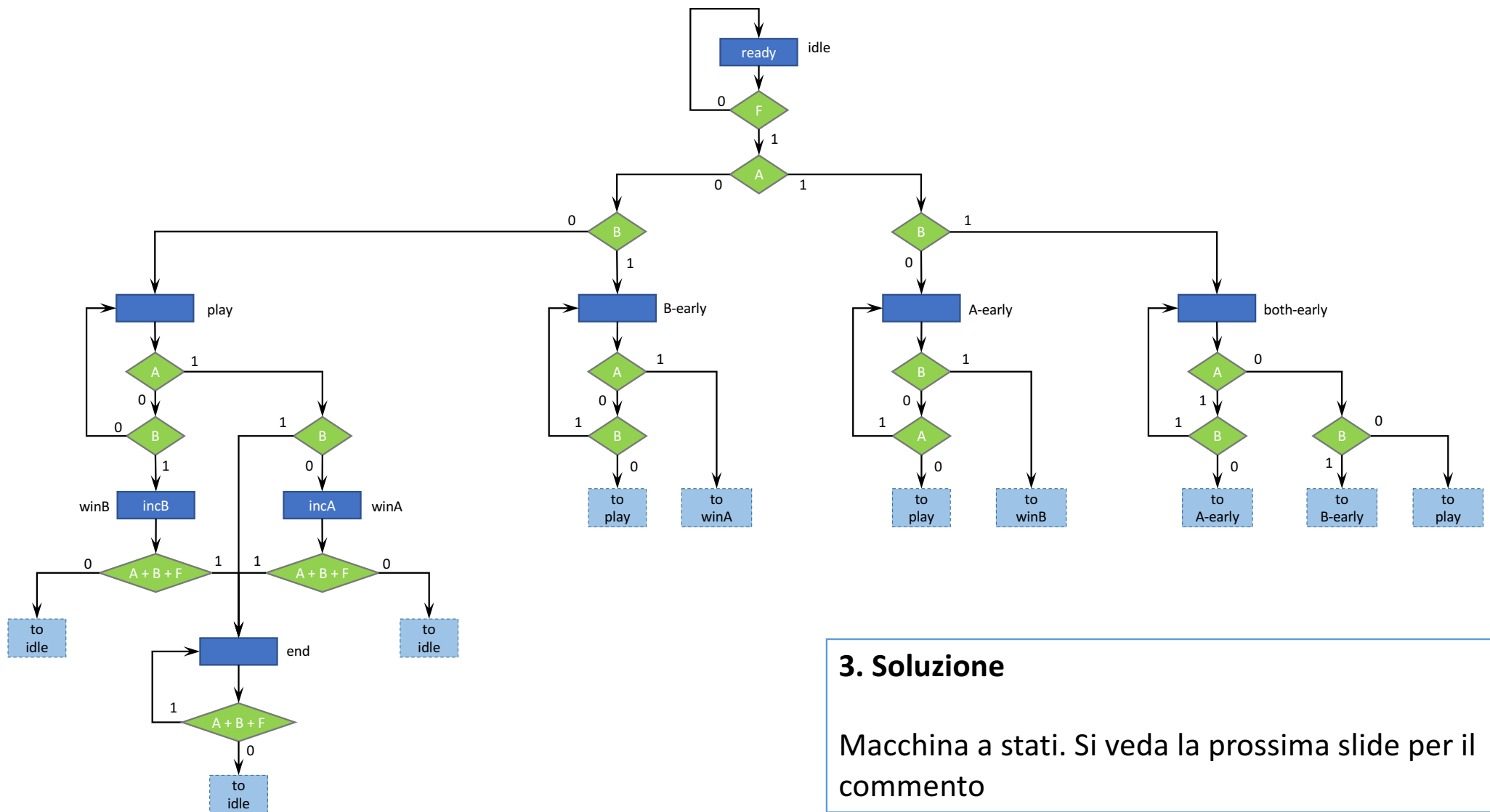
- I tempi relativi tra i segnali **A**, **B** ed **F** all'interno di un ciclo di clock non devono essere presi in considerazioni. Ci interessa solo il valore dei segnali al fronte del clock. Tutti i segnali sono sincroni con il clock
- Il segnale **F** può stare a 1 per uno o più cicli di clock. La giocata comincia al fronte in cui lo si vede andare a 1
- Poiché vi potranno essere molte transizioni, in alcuni casi sarà utile indicare semplicemente il nome dello stato destinazione della transizione, senza necessariamente portare la freccia allo stato corrispondente

Obiettivo del gioco

- Due giocatori **A** e **B** si sfidano in una gara di riflessi. Il gioco consiste nell'essere più veloci a premere un pulsante in seguito all'attivazione di un flash **F** da parte di un terzo partecipante neutrale. Il sistema deve tenere traccia dei punti collezionati da ciascun giocatore. Dobbiamo sviluppare l'unità MSF.

Regole del gioco e funzionamento

- Inizialmente si suppone che i segnali **A**, **B** ed **F** siano a 0, ed il segnale **ready** indica che si è pronti per la prossima giocata
- Da questo momento i segnali **A**, **B** ed **F** possono andare a 1 in qualunque ciclo di clock (anche contemporaneamente)
- La giocata ha inizio quando **F** va a 1, e la macchina a stati pone il segnale **ready** a 0
- In una giocata normale, i pulsanti **A** e **B** saranno a 0 quando **F** va a 1. In tal caso la MSF attende il primo pulsante che viene premuto, ed incrementa il corrispondente contatore per assegnare il punto. Se **A** e **B** vengono premuti contemporaneamente, la giocata è pari e non si assegna nessun punto
- Se un giocatore ha il pulsante già premuto quando arriva il flash **F** (e.g, **A** = 1 nello stesso ciclo in cui **F** va a 1), affinché la sua giocata possa essere considerata valida, *deve prima rilasciare il pulsante e quindi ripremere*lo. Nel frattempo l'altro giocatore può invece fare la sua giocata normalmente
- Se entrambi i giocatori hanno il pulsante già premuto quando arriva il flash **F**, la regola precedente vale per entrambi, e vincerà il primo che rilascia e quindi ripreme il pulsante
- In tutti i casi, una volta terminata la giocata il sistema attende che tutti i segnali **A**, **B** ed **F** ritornino a 0 prima di fornire il segnale **ready** ed essere pronto per una nuova giocata (il segnale **ready** era stato disattivato all'arrivo di **F**)



Stato idle

- In questo stato si attende che **F** vada a 1 per cominciare il gioco.
- Quando **F** = 1, si osservano anche i valori di **A** e **B**, per verificare se i pulsanti sono stati premuti prima del dovuto. Sulla base dei confronti si esegue una transizione in uno dei prossimi quattro stati.

Stato play

- I pulsanti non erano premuti e si procede con una giocata normale. Si guardano **A** e **B**, il primo che viene premuto porta nello stato che incrementa il contatore corrispondente.
- Se entrambi sono premuti contemporaneamente, non si assegnano punti e si salta direttamente in **end**.

Stati A-early e B-early

- Si va in **A-early** se il pulsante **A** era premuto quando **F** = 1, mentre **B** no. Guardiamo allora **B** ed **A**. Se viene premuto **B** vince B, altrimenti si guarda se **A** è ancora premuto. Se è stato rilasciato si passa a **play** per continuare la giocata.
- Lo stato **B-early** è simmetrico.

Stato both-early

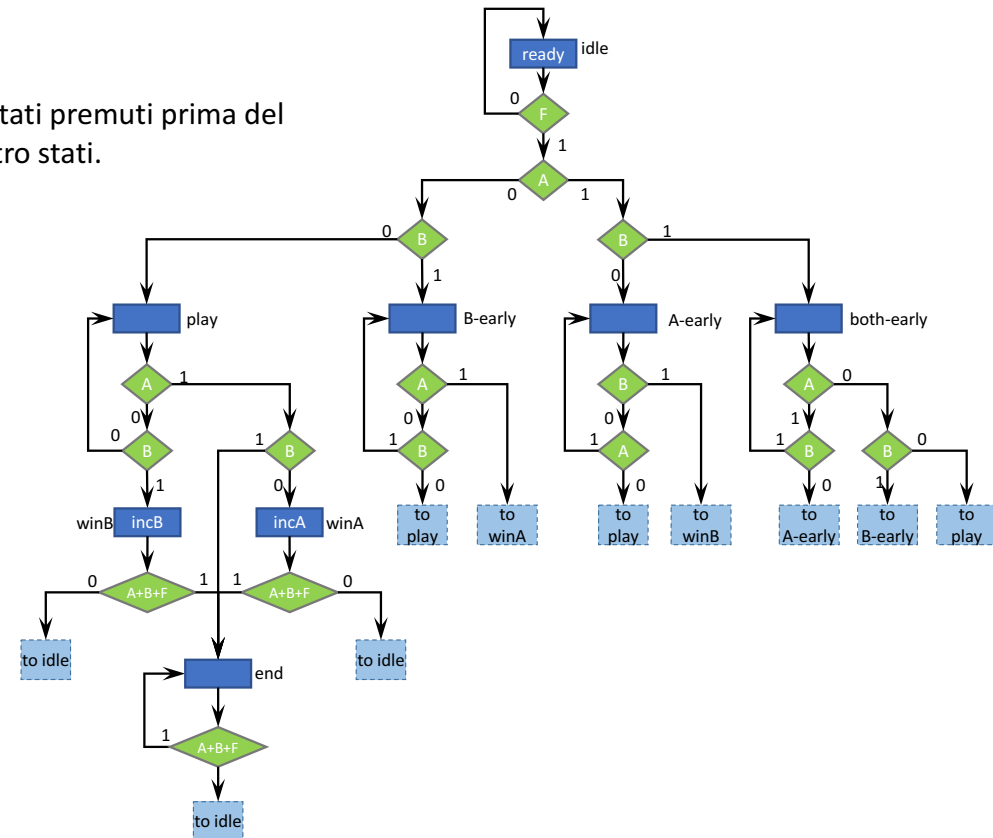
- Entrambi premuti quando **F** è andato a 1. Si aspetta che uno dei due venga rilasciato, andando nello stato **x-early** appropriato, o in **play** se vengono rilasciati entrambi.

Stati winA e winB

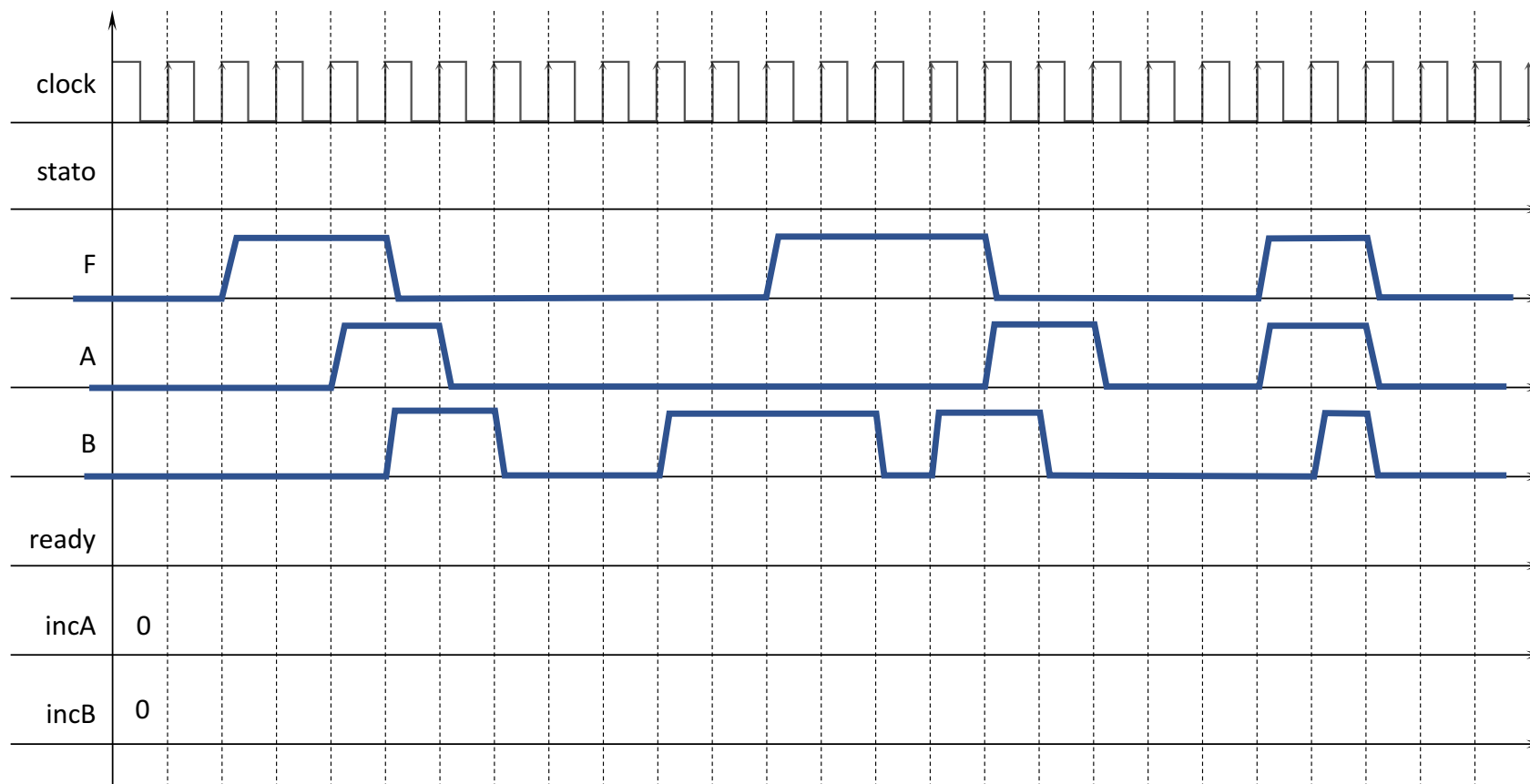
- Si incrementa il contatore. Si guarda se i segnali sono già tutti a 0, nel qual caso si torna in **idle** (era considerato corretto anche non fare questa verifica). Altrimenti si va in **end** che attende che i segnali tornino a 0. Si potrebbero avere uscite condizionate al posto di questi stati per ridurre il numero degli stati.

Stato end

- Attende che tutti i segnali vadano a 0. Si usa un solo blocco condizionale con una espressione per evitare di avere una condizione su ogni segnale (ma anche con una verifica per segnale va bene).

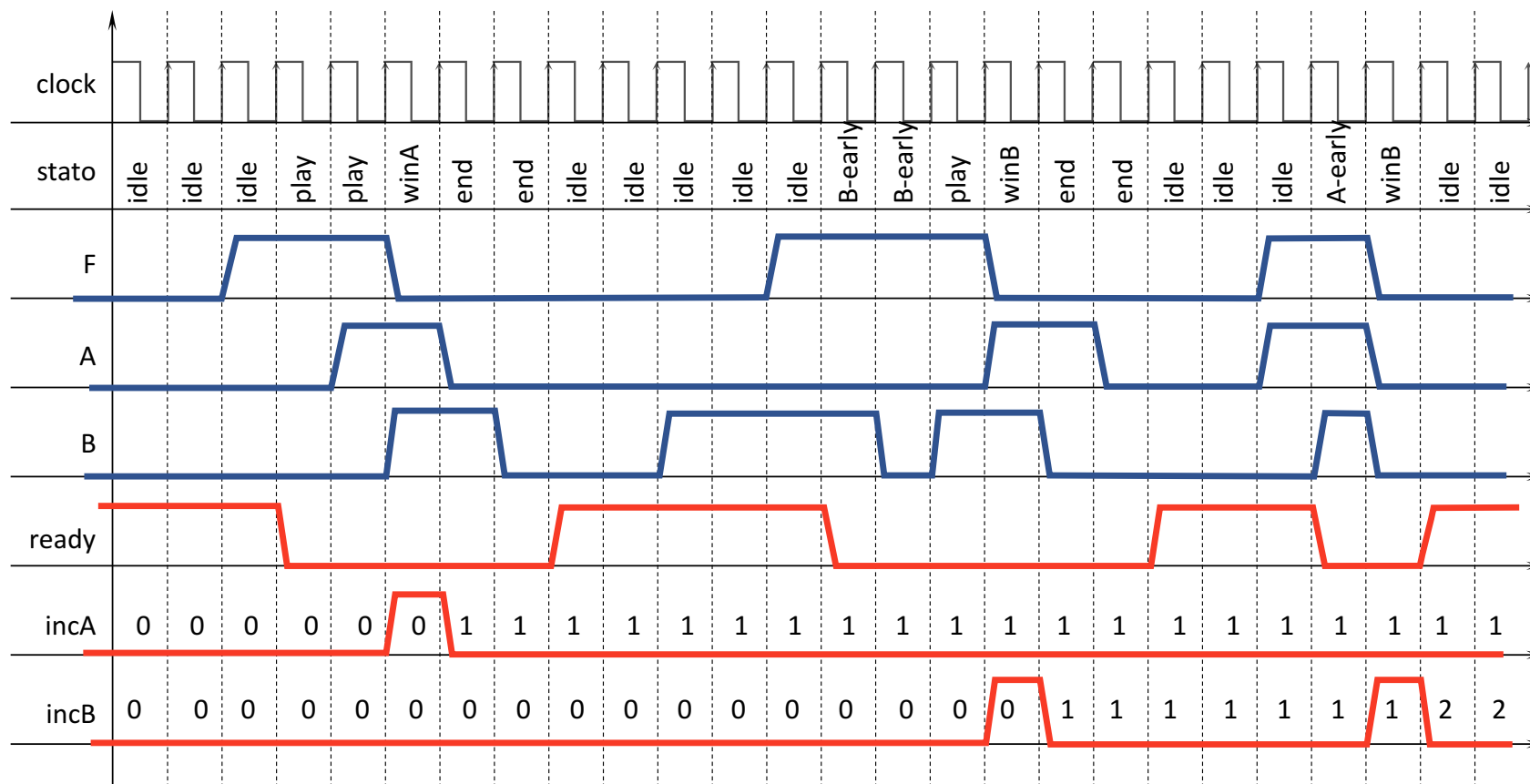


Nome e matricola:



Nota: indicare nelle righe incA ed incB sia il valore del segnale, sia il valore numerico del contatore

Nome e matricola:



Nota: indicare nelle righe incA ed incB sia il valore del segnale, sia il valore numerico del contatore